

Programowanie i metody numeryczne

Ćwiczenia 1.

Podstawy programowania – powtórzenie.

Zadanie 1. Kula trójwymiarowa.

Standardowe wejście, standardowe wyjście, argumenty wywołania.

W przypadku trójwymiarowej kuli o promieniu R prawdziwe są następujące wzory:

$$\begin{aligned}\text{pole powierzchni: } P &= 4\pi R^2, \\ \text{objętość: } V &= \frac{4}{3}\pi R^3.\end{aligned}$$

- a) Napisz program **balli**, który **wczytuje ze standardowego wejścia** liczbę rzeczywistą R , a następnie oblicza i wypisuje na standardowe wyjście pole powierzchni i objętość trójwymiarowej kuli o promieniu R .
- b) Napisz program **balla**, który **przyjmuje jako argument wywołania** liczbę rzeczywistą R , a następnie oblicza i wypisuje na standardowe wyjście pole powierzchni i objętość trójwymiarowej kuli o promieniu R .

Zadanie 2. Liczby parzyste i liczby nieparzyste.

Instrukcje warunkowe, pętle.

Napisz program **evenodd**, który przyjmuje jako argumenty wywołania dwie liczby całkowite m i n , a następnie wypisuje na standardowe wyjście kolejne liczby całkowite od m do n włącznie, każdą w osobnej linii, podając przy każdej z nich informację o tym, czy jest ona parzysta, czy nieparzysta.

Zadanie 3. Wskaźnik masy ciała.

Funkcje.

Wskaźnik masy ciała (ang. *Body Mass Index*, BMI) to współczynnik stosowany do określania poprawności masy ciała. Oblicza się go zgodnie ze wzorem

$$\text{BMI} = \frac{\text{masa ciała w kilogramach}}{(\text{wzrost w metrach})^2}.$$

W przypadku młodej osoby dorosłej wartości BMI interpretowane są w następujący sposób:

- poniżej 18,5 – niedowaga,
- 18,5 – 25 – waga prawidłowa,
- 25 – 30 – nadwaga,
- powyżej 30 – otyłość.

- a) Napisz funkcję `bmi_value`, która przyjmuje jako argumenty dwie liczby rzeczywiste, masę ciała w kilogramach oraz wzrost w metrach, oraz zwraca wartość BMI odpowiadającą tym danym.
- b) Napisz funkcję `bmi_interpretation`, która przyjmuje jako argument liczbę rzeczywistą reprezentującą wartość BMI i zwraca łańcuch tekstowy będący interpretacją tej wartości zgodnie z powyższymi kryteriami.
- c) Korzystając z tych funkcji, napisz funkcję `bmi`, która przyjmuje jako argumenty dwie liczby rzeczywiste, masę ciała w kilogramach oraz wzrost w metrach, oraz zwraca dwie wartości: wartość BMI odpowiadającą tym danym oraz jej interpretację.
- d) Korzystając z funkcji `bmi`, napisz program `bmi`, który wczytuje ze standardowego wejścia masę ciała w kilogramach i wzrost w metrach, a następnie wypisuje na standardowe wyjście wartość BMI odpowiadającą tym danym oraz jej interpretację.

Przykładowe wykonanie

Wejście

Podaj masę ciała (w kilogramach): 70

Podaj wzrost (w metrach): 1.8

Wyjście

BMI = 21.60

Waga prawidłowa.

Zadanie 4. Skala Fahrenheita.

Funkcje lambda.

Temperaturę t_F wyrażoną w stopniach Fahrenheita można obliczyć na podstawie temperatury t_C wyrażonej w stopniach Celsjusza, korzystając z następującego wzoru:

$$t_F = \frac{9}{5}t_C + 32.$$

Napisz program `fahrenheit`, który wczytuje ze standardowego wejścia listę oddzielonych spacjami liczb rzeczywistych reprezentujących temperatury w stopniach Celsjusza, a następnie wypisuje na standardowe wyjście, w kolejnych liniach, te temperatury i odpowiadające im temperatury w stopniach Fahrenheita. Program powinien wykorzystywać funkcję `lambda` do przekształcenia temperatury z jednej skali na drugą.

Wskazówka 1. Wykorzystaj metodę `split` do zastąpienia łańcucha tekstowego zwracanego przez funkcję `input` listą zawartych w nim słów.

Wskazówka 2. W celu zaaplikowania funkcji `lambda` do wszystkich elementów listy możesz wykorzystać funkcję `map`.

Zadanie 5. Obliczanie liczby π .

Funkcje, wizualizacja danych.

Jednym ze sposobów obliczania wartości liczby π jest wykorzystanie tzw. *wzoru Machina*:

$$\frac{\pi}{4} = 4 \arctg \frac{1}{5} - \arctg \frac{1}{239}$$

w połączeniu z rozwinięciem funkcji *arcus tangens* w szereg Maclaurina:

$$\arctg x = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} x^{2n+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots, \quad \text{gdzie } |x| \leq 1.$$

- a) Napisz funkcję `arctg_series`, która przyjmuje jako argumenty liczbę rzeczywistą x z przedziału $[-1, 1]$ oraz liczbę naturalną N i zwraca wartość funkcji $\arctg x$ obliczoną na podstawie jej rozwinięcia w szereg Maclaurina, biorąc N pierwszych wyrazów tego szeregu.
- b) Korzystając z funkcji `arctg_series`, napisz funkcję `pi_estimate`, która przyjmuje jako argument liczbę naturalną N i zwraca oszacowanie liczby π obliczone na podstawie wzoru Machina, biorąc N pierwszych wyrazów rozwinięcia funkcji *arcus tangens* w szereg Maclaurina.
- c) Wykorzystując funkcję `pi_estimate`, napisz program `pi`, który przyjmuje jako argument wywołania liczbę całkowitą K , a następnie oblicza na podstawie wzoru Machina liczbę π , biorąc N pierwszych wyrazów rozwinięcia funkcji *arcus tangens*, dla $N = 1, 2, \dots, K$. Program powinien narysować wykres otrzymanych wartości liczby π w funkcji N , dla porównania nanieść na nim prostą $y = \pi$, zaś w tytule wykresu umieścić oszacowanie π dla $N = K$.

Zadanie 6. Sklep z elektroniką.

Kolekcje: listy.

Napisz program `store`, którego zadaniem jest analiza listy zamówień złożonych w sklepie internetowym.

Program powinien na początku wczytać ze standardowego wejścia ciąg dowolnie wielu łańcuchów tekstowych, z których każdy pochodzi ze zbioru:

`laptop, smartphone, tablet, headphones, smartwatch, camera, printer, monitor,`

oddzielonych spacjami, oraz utworzyć listę `orders` zawierającą elementy tego ciągu w podanej kolejności. Możesz założyć, że wczytany ciąg będzie zawierał co najmniej pięć łańcuchów tekstowych.

Następnie program ma wykonać następujące operacje, unikając stosowania pętli i instrukcji warunkowych (pamiętaj o tym, że indeksowanie elementów listy zaczyna się od zera):

- a) wypisać na standardowe wyjście łączną liczbę zamówień, czyli długość listy `orders`, oraz pierwszy (indeks 0), trzeci (indeks 2) i ostatni (indeks -1) element listy `orders`,
- b) wypisać na standardowe wyjście pierwsze trzy elementy i ostatnie trzy elementy listy `orders`,
- c) dopisać na koniec listy `orders` trzy kolejne zamówienia: `laptop`, `smartphone` i `headphones`, a następnie wypisać na standardowe wyjście nową długość listy `orders` oraz jej ostatni element,
- d) wypisać na standardowe wyjście co drugi element listy `orders`, począwszy od drugiego elementu listy (czyli elementu o indeksie 1), a następnie co drugi element listy `orders` w odwróconej kolejności, począwszy od przedostatniego elementu listy (czyli elementu o indeksie -2),
- e) wypisać na standardowe wyjście informację o tym, ile razy każdy z produktów został zamówiony,
- f) utworzyć listę `unique_orders`, zawierającą elementy listy `orders`, jednak każdy tylko raz (bez powtórzeń), posortować ją alfabetycznie, a następnie wypisać ją na standardowe wyjście,
- g) utworzyć trzy nowe listy:
 - `big_orders`, zawierającą tylko te elementy listy `orders`, które wystąpiły na niej co najmniej trzy razy,
 - `small_orders`, zawierającą tylko te elementy listy `orders`, które wystąpiły na niej tylko raz,
 - `special_orders`, będącą połączeniem list `big_orders` i `small_orders`,
 a następnie wypisać na standardowe wyjście listę `special_orders`,
- h) usunąć z listy `orders` wszystkie wystąpienia produktu `smartphone`, a następnie wypisać na standardowe wyjście nową długość listy `orders`.

Zadanie 7. Dziennik ocen.

Kolekcje: słowniki.

W jednej z klas pewnej szkoły podstawowej uczniowie otrzymali następujące oceny na koniec roku szkolnego:

Uczeń	Alicja	Bartłomiej	Celina
Matematyka	3	5	4
Język polski	4	4	5

Napisz program `gradebook` będący prostym dziennikiem ocen. Program powinien:

- tworzyć słownik `grades`, w którym kluczami będą imiona uczniów, zaś wartościami – zagnieżdżone słowniki, w których kluczami będą nazwy przedmiotów, a wartościami – oceny z tych przedmiotów, wypełniając ten słownik danymi z powyższej tabeli,
- dodać do słownika `grades` ucznia o imieniu Daniel, który otrzymał ocenę 4 z matematyki i ocenę 3 z języka polskiego,
- zmienić ocenę Bartłomieja z języka polskiego na 5,
- wypisać na standardowe wyjście imiona i średnie ocen wszystkich uczniów, posortowane malejąco według średniej ocen,
- wypisać na standardowe wyjście imiona uczniów, którzy otrzymali ocenę 5 z matematyki, oraz imiona uczniów, którzy otrzymali ocenę 5 z języka polskiego,
- wypisać na standardowe wyjście średnią ocen z matematyki wszystkich uczniów oraz średnią ocen z języka polskiego wszystkich uczniów,
- usunąć ze słownika `grades` Celinę, a następnie ponownie wypisać na standardowe wyjście średnią ocen z matematyki wszystkich uczniów oraz średnią ocen z języka polskiego wszystkich uczniów.

Zadanie 8. Słowa.

Wyrażenia listotwórcze (list comprehension) i słownikotwórcze (dictionary comprehension).

Napisz program `words`, który tworzy listę `words` zawierającą następujące łańcuhy tekstowe:

`kot, komputer, kajak, pies, programowanie, oko, python, ala, rower.`

Następnie program powinien utworzyć **bez korzystania z pętli** i wypisać na standardowe wyjście:

- listę zawierającą te elementy listy `words`, których długość jest większa niż 3 znaki,
- listę zawierającą te elementy listy `words`, które są palindromami (czyli łańcuchami tekstowymi, które czytane od tyłu są takie same, jak czytane od przodu),
- słownik, którego kluczami są elementy listy `words`, a wartościami – ich długości,
- słownik grupujący elementy listy `words` według ich pierwszej litery, tzn. słownik, którego kluczami są litery, a wartościami – listy zawierające te elementy listy `words`, które zaczynają się na daną literę.

Zadanie 9. Ciąg Collatza.

Kolekcje.

Niech k będzie liczbą naturalną. *Ciągiem Collatza* nazywamy ciąg $(c_n^k)_{n=0}^\infty$ określony wzorem

$$c_n^k \stackrel{\text{def}}{=} \begin{cases} k, & \text{gdy } n = 0, \\ \frac{1}{2}c_{n-1}, & \text{gdy } c_{n-1} \text{ jest liczbą parzystą,} \\ 3c_{n-1} + 1, & \text{gdy } c_{n-1} \text{ jest liczbą nieparzystą.} \end{cases}$$

Na przykład

$$\begin{aligned} (c_n^{12}) &= (12, 6, 3, 10, 5, 16, 8, 4, 2, \mathbf{1}, 4, 2, 1, 4, 2, 1, \dots), \\ (c_n^{13}) &= (13, 40, 20, 10, 5, 16, 8, 4, 2, \mathbf{1}, 4, 2, 1, 4, 2, 1, \dots), \\ (c_n^{15}) &= (15, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, \mathbf{1}, 4, 2, 1, 4, 2, 1, \dots). \end{aligned}$$

Gdy pewien wyraz ciągu Collatza ma wartość 1, wyrazy następujące po nim mają wartości 4, 2, 1, 4, 2, 1, ... – sekwencja 4, 2, 1 powtarza się w nieskończoność. Przypuszcza się, że ciąg Collatza (c_n^k) osiąga wartość 1 dla dowolnego k , hipoteza ta pozostaje jednak problemem otwartym.

- Napisz program `collatz`, który prosi użytkownika o podanie liczby naturalnej k , a następnie oblicza i wypisuje początkowe wyrazy ciągu Collatza (c_n^k) aż do pierwszego z wyrazów o wartości 1 włącznie.
- Napisz program `lcollatz` znajdujący tę liczbę naturalną $k < 10^6$, dla której ciąg Collatza (c_n^k) ma najwięcej wyrazów występujących przed pierwszym z wyrazów o wartości 1.

Zadanie 10. coviddata – Krzywa zachorowań na COVID-19.

Kolekcje, wizualizacja danych.

Napisz program `coviddata`, wykreślający krzywą zachorowań na COVID-19 dla wybranej przez użytkownika lokalizacji. Program powinien pobierać z internetu plik z danymi dotyczącymi ilości zachorowań w formacie CSV, udostępniony pod adresem

https://raw.githubusercontent.com/owid/covid-19-data/refs/heads/master/public/data/jhu/full_data.csv

a następnie wypisywać listę lokalizacji – krajów, kontynentów i całego świata – dla których dane są dostępne, prosić użytkownika o wskazanie jednej z nich i rysować krzywą przedstawiającą dzienną ilość nowych zachorowań na COVID-19 w tej lokalizacji.

Szkic rozwiązania.

- Napisz kod pobierający plik z danymi i zapisujący go w bieżącym katalogu. Możesz użyć do tego celu na przykład funkcji `get` z modułu `requests` (prościej, wymagana wcześniejsza instalacja pakietu `requests`) lub funkcji `urlopen` z modułu `urllib.request` (trudniej, jednak nie wymaga instalacji).
- Wczytaj plik z danymi, używając polecenia

```
lines = open("full_data.csv").readlines()
```

Jakiego typu jest obiekt `lines`? Możesz to sprawdzić korzystając z funkcji `type()`.

- Zapisz w zmiennej `data` trzeci element listy `lines`. Jakiego typu jest obiekt `data`?

- d) Podziel łańcuch `data` na fragmenty z przecinkiem jako separatorem, wywołując metodę `split()` z odpowiednimi argumentami. Zbadaj, jak wygląda wynik działania tej funkcji i wydobydź z niego informacje o dacie, kraju oraz ilości zachorowań. Wypisz te dane, by sprawdzić, czy kod działa poprawnie.
- e) Zmodyfikuj swój kod tak, by zamiast analizować pojedynczą linię pliku z danymi, badał wszystkie.

Można w tym celu po prostu analizować kolejne elementy listy `lines` (np. za pomocą pętli `for`). Rozwiązanie takie byłoby poprawne, miałyby jednak istotną wadę. Polecenie `open("full_data.csv").readlines()` wczytuje do pamięci od razu cały plik `full_data.csv` i tworzy ogromną listę `lines` zawierającą wszystkie jego linie. Proces ten w przypadku dużej ilości danych jest długotrwały i zużywa sporo pamięci operacyjnej; co więcej, gdyby plik z danymi był zbyt duży, pamięci mogłoby zabraknąć – zostałby wówczas zgłoszony wyjątek `MemoryError`, a program nie wykonałby swojego zadania.

Bezpieczniej jest wczytywać plik z danymi linia po linii, program będzie wówczas działał poprawnie dla dowolnie dużej ilości danych. Można w tym celu posłużyć się na przykład następującym kodem (zamiast tworzyć listę `lines`):

```
with open("full_data.csv") as datafile:
    for line in datafile:
        # Łańcuch line zawiera treść pojedynczej linii pliku.
```

Zastosowanie polecenia `with` daje nam pewność, że po zakończeniu pracy z plikiem wszystkie związane z nim zasoby zostaną zwolnione (wcześniej nie przejmowaliśmy się tym problemem, co mogło skutkować pojawieniem się tzw. wycieku pamięci).

Utwórz pusty słownik o nazwie `cases_pol`, a następnie przeanalizuj kolejne linie pliku z danymi, sprawdzając, czy dana linia dotyczy Polski, i – jeśli tak jest – dodaj do słownika `cases_pol` element, którego kluczem będzie data (np. w postaci łańcucha znaków), zaś wartością – liczba zachorowań. Narysuj wykres danych zawartych w słowniku `cases_pol`, by sprawdzić, czy Twój kod działa poprawnie.

Postaraj się tak skonfigurować wykres, by był on czytelny i estetyczny. W celu zapewnienia poprawnego wyświetlania etykiet na osi odciętych konieczne może się okazać przechowywanie dat w słowniku `cases_pol` nie w postaci łańcuchów tekstowych, lecz za pomocą obiektów dedykowanego datom typu, np. typu `datetime` zdefiniowanego w module `datetime`.

- f) Zmodyfikuj program tak, by analizował dane o zachorowaniach dla wybranej przez użytkownika lokalizacji.

Najprostszym sposobem realizacji tego celu wydaje się być zastąpienie słownika `cases_pol` słownikiem `cases` mającym strukturę „słownika słowników”: każdy jego klucz powinien być nazwą jednej z lokalizacji, dla których dane o zachorowaniach są dostępne, zaś wartość odpowiadająca danemu kluczowi – słownikiem zawierającym dane o zachorowaniach w lokalizacji, na którą wskazuje ten klucz, zbudowanym analogicznie, jak omówiony szczegółowo w poprzednim punkcie słownik `cases_pol`. Obiekt `cases["Italy"]` będzie wówczas słownikiem zawierającym dane o zachorowaniach we Włoszech, obiekt `cases["Poland"]` – słownikiem z danymi o zachorowaniach w Polsce (dokładnie tym samym, którym wcześniej był `cases_pol`) etc.

Rozwiązanie to, choć poprawne, nie jest optymalne, w pamięci operacyjnej powstanie bowiem ogromna struktura danych, przechowująca wszystkie informacje zawarte w pliku z danymi. Gdy informacji tych będzie dużo, mogą wystąpić te same problemy, o których wspominaliśmy w poprzednim punkcie.

Lepszym rozwiązaniem będzie przeprowadzenie analizy danych dwuetapowo. Na początku możemy wydobyć z pliku z danymi wyłącznie informacje o tym, dla jakich lokalizacji dostępne są dane, następnie poprosić użytkownika o wskazanie interesującej go lokalizacji i ponownie przejrzeć plik, odczytując informacje o zachorowaniach wyłącznie dla tej lokalizacji.

Kierując się tą ideą, zmodyfikuj kod programu w następujący sposób. Zamiast słownika `cases_pol` program powinien tworzyć pusty zbiór o nazwie `locations`, a potem analizować kolejne linie pliku z danymi, wydobywając z każdej z nich informację o tym, jakiej lokalizacji dotyczą zawarte w niej

dane, i dodawać nazwę tej lokalizacji do zbioru `locations` (zastosowanie zbioru będzie tu sporym ułatwieniem – zbiór, w przeciwieństwie do np. listy, utożsamia ze sobą identyczne elementy, nie pozwalając na pojawienie się duplikatów). Program powinien następnie wypisać elementy zbioru `locations` i poprosić użytkownika o wybranie jednego z nich. Gdy użytkownik poda poprawną nazwę lokalizacji (np. `Poland`), program powinien utworzyć słownik o nazwie `cases` zawierający dane na temat zachorowań w tej lokalizacji (skonstruowany analogicznie, jak omówiony szczegółowo w poprzednim punkcie słownik `cases_pol`) i – na jego podstawie – narysować krzywą zachorowań dotyczącą tej lokalizacji.

Opracowanie: Bartłomiej Zglinicki.